

Linked List

=====

REMEMBER 1: On the top of every program, including the following comment header:

```
/* Name:           Your Name

   Student Code:   Your Student Code

   Purpose:        A short description of the program/exercise.

*/
```

REMEMBER 2: You must include a **README.txt** file. The file contains instructions on how to compile and run your code.

=====

1. Introduction:

You have learnt about Linked List data structure. This lab will help you go through some practice manners of Linked List data in Java programming language.

2. Problems:

Problem I: Circular Linked list

A circular list is a linked list in which the last link points back to the first link. There are many ways to design a circular list. Sometimes there is a pointer to the "**start**" of the list. However, this makes a list less like a real circle and more like an ordinary list with its end attached to its beginning.

Make a class for a singly linked circular list with no end or beginning. The only access to the list is a single reference, **current**, that can point to any link on the list. This reference can move around the list as needed. (See next problem for a situation in which such a circular list is ideally suited.).

Your list should handle **insertion**, **searching**, and **deletion**. You may find it convenient if these operations take place one link downstream of the link pointed to by **current**. (Because the upstream link is singly linked, you can't get at it without going around the circle.)

You should also be able to display the list (although you'll need to break the circle at some arbitrary point to print it on the screen). A **step()** method that moves **current** along to the next link might come in handy too.

=====

Problem II: Josephus Problem (https://en.wikipedia.org/wiki/Josephus_problem)

The Josephus Problem is a famous mathematical puzzle that goes back to ancient times. There are many stories to go with the puzzle. One is that Josephus was one of a group of Jews who were about to be captured by the Romans. Rather than be enslaved, they chose to commit suicide. They arranged themselves in a circle and, starting at a certain person, started counting off around the circle. Every n th person had to leave the circle and commit suicide. Josephus decided he didn't want to die, so he arranged the rules so he would be the last person left. If there were (say) 41 people, and he was the 16th person from the start of the circle, what number should he tell them to use for counting off? The problem is made much more complicated because the circle shrinks as the counting continues.

The problem—given the number of people, starting point, direction, and number to be skipped—is to choose the position in the initial circle to avoid execution.

Task:

- Create an application that uses a circular linked list (like that in Programming Project 5.3) to model this problem.
- The application must ask for user inputs:
 - o the number of people in the circle
 - o the number used for counting off
 - o the number of the person where counting starts (usually 1).
- The application must print out the output: the list of people being eliminated in order (assuming you go around clockwise).

See Figure 1 below for a visualization of an example. There are 41 soldiers numbered 1 through 41. They start counting at 1 and with a step size of three. The last two soldiers remaining are #16 and #31. **Note:** *if the counting continues with those two soldiers, then the last one remaining is #31 only.* Figure 2 shows the example input and output that your application should have.

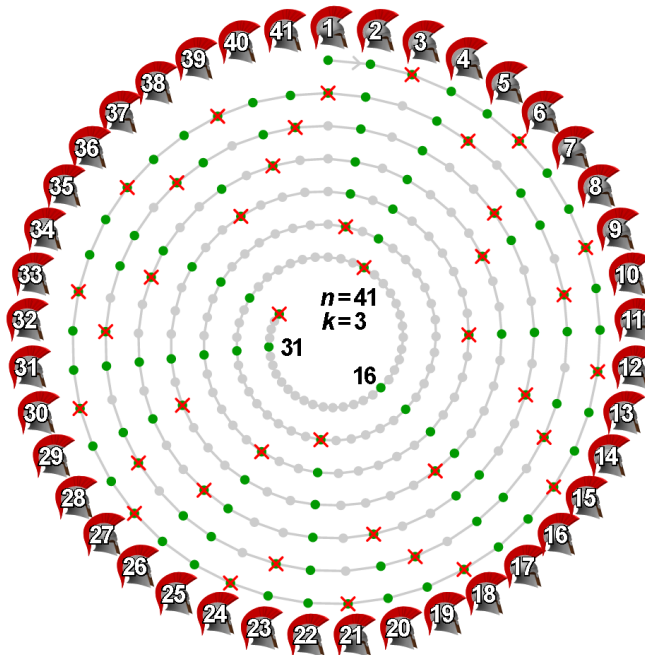


Figure 1: Claude Gaspar Bachet de Méziriac's interpretation of the Josephus problem with 41 soldiers and a step size of 3, showing that places 16 and 31 are last to be killed – time progresses inwards along the spiral, green dots denoting live soldiers, grey dead soldiers, and crosses killings.

```

Enter the number of people in the circle: 41
Enter the number used for counting off: 3
Enter the number of the person where counting starts: 1
Elimination order:
3 6 9 12 15 18 21 24 27 30 33 36 39 1 5 10 14 19 23 28 32 37 41 7 13 20
26 34 40 8 17 29 38 11 25 2 22 4 35 16
Last person standing: 31

```

Figure 2: Example input and output

=====

Problem 3: Doubly Linked list

Complete and submit the in-class lab assignment

Download file ***DoublyLinkedApp.java***

Complete the functions:

- insertFirst

- insertLast
- deleteFirst
- deleteLast
- deleteKey
- displayForward
- displayBackward